Universidad Nacional Rosario Castellanos

Licenciatura en Ciencia de Datos para Negocios

Unidad Académica Chalco

Tercer semestre · Semestre 2026-2

Problema prototípico: Bacheo inteligente y priorización urbana

SIPA

Sistema Inteligente de Priorización Asfáltica

Segundo avance · Estructuras de Datos

Prototipo funcional y plan de viabilidad

#	Integrantes del equipo
1	Sandra Luna Altos
2	Joshua David Aquino
3	Rebeca Jaimes Cruz
4	Ithan Felipe Santiago

Dato	
UCA	Estructuras de Datos (LCDN3EST8)
Docente	Luisa Reyna Beatriz Ayon Muñoz
Grupo	302
Fecha de entrega	13 de noviembre de 2026

Declaración de uso de IA. El código del prototipo (carpeta estructuras/) y este documento se elaboraron con apoyo de IA generativa (Claude, de Anthropic). El equipo revisó, ejecutó y probó el código, y es responsable de su contenido. Todas las estructuras están programadas desde cero, sin heapq, dict de Python como tabla hash ni otras bibliotecas que ya las resuelvan.

1. El problema: una cola FIFO para atender baches

El incidente crítico describe un sistema que atiende los reportes en orden de llegada (FIFO). Eso trae tres fallas:

- Ignora la urgencia. En nuestra simulación de 90 días de lluvias de 2019 con datos reales, la cola FIFO atiende a tiempo (3 días o menos) solo el 6 % de los baches de mayor riesgo.
- Satura la memoria con duplicados: varios vecinos reportan el mismo bache y cada aviso se vuelve una solicitud distinta.

- Genera rutas caóticas: la brigada visita los baches en el orden en que llegaron, sin importar dónde están.

El prototipo sustituye la cola por un conjunto de estructuras, cada una para una parte del problema. Un reporte recorre este camino:

Paso	Estructura	Complejidad
1. Llega el reporte	Lista doblemente enlazada	O(1)
2. ¿Ya existe ese bache?	Tabla hash (y conjunto)	O(1) en promedio
3. Se calcula su prioridad y entra a la cola de atención	Max-heap	O(log n)
4. La autoridad ajusta una prioridad y puede deshacerlo	Pila de deshacer y rehacer	O(1) + O(log n)
5. Se arma el plan del día y la ruta de la brigada	Max-heap + grafo con Dijkstra	O(k log n) + O((V+E) log V)
6. Consultas por rango (por ejemplo, una zona)	Árbol AVL	O(log n + k)

Datos: los reportes reales de baches de la CDMX 2019-2022 (sin periodos atípicos) y el riesgo de accidente a 72 horas por celda que calcula el modelo de Probabilidad. En lluvias llegan 37.3 baches nuevos por día y en secas 22.5.

2. Diseño orientado a objetos

El dominio se modela con tres clases. Reporte guarda el aviso ciudadano y calcula su clave de ubicación. Bache agrupa los reportes de un mismo daño y calcula su prioridad. Brigada recibe un plan y arma su ruta sobre el grafo. La clase SistemaSIPA integra las estructuras: cada una es un atributo con una sola responsabilidad (encapsulamiento), y sus operaciones se usan sin conocer cómo están programadas por dentro.

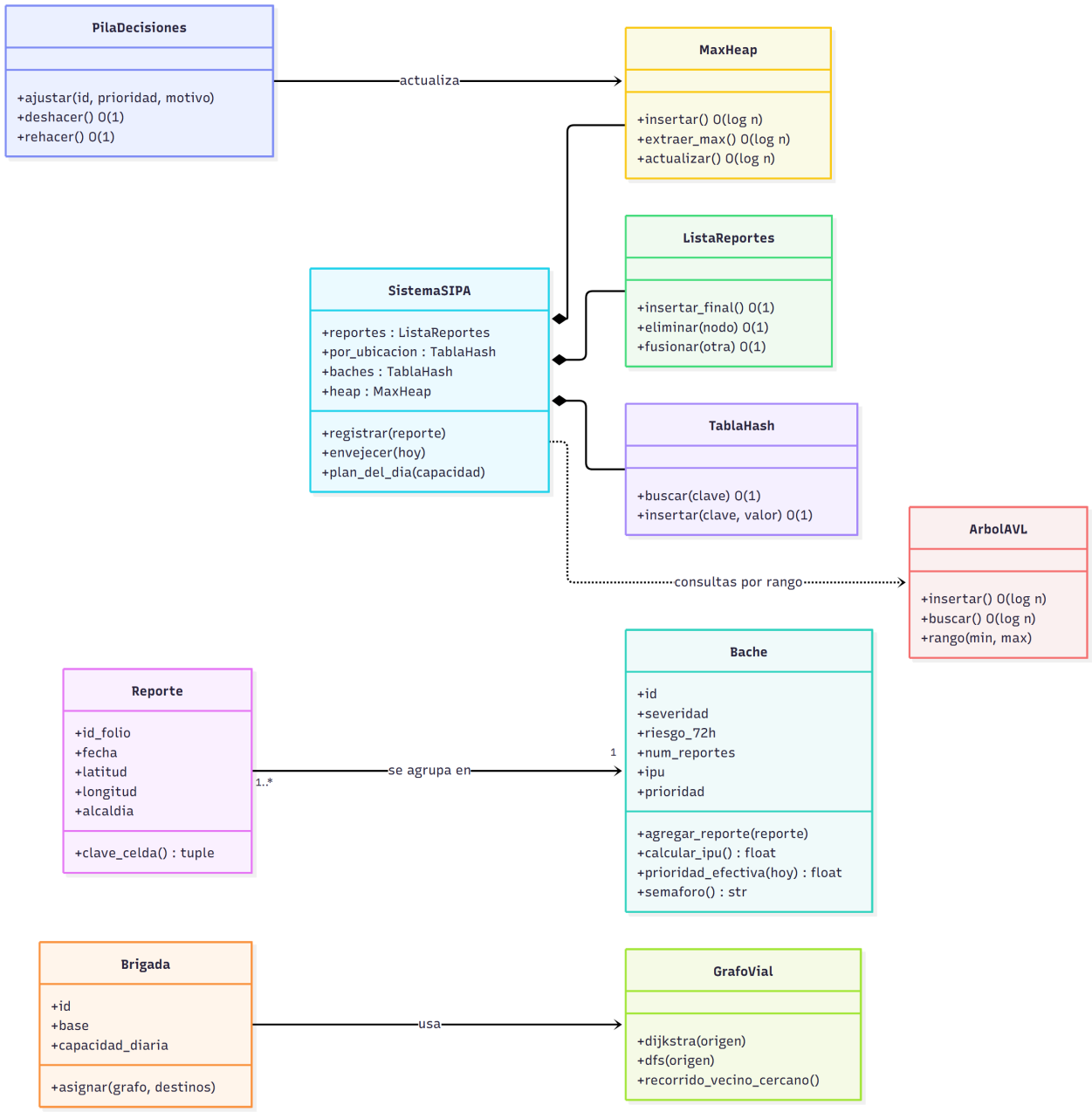


Figura 1. Diagrama de clases del prototipo (estructuras/.py).*

El índice de prioridad de un bache vive en un método de la clase, así que cambiar la fórmula no afecta a ninguna estructura:

```
def calcular_ipu(self):
    v_sev = (self.severidad - 1) / 4
    v_rep = min(math.log(1 + self.num_reportes) / math.log(1 + TOPE_REPORTES), 1.0)
    return round(PESO_SEVERIDAD * v_sev + PESO_RIESGO * self.riesgo_72h + PESO_REPORTES * v_rep, 4)
```

modelos.py · Bache.calcular_ipu: severidad (45 %), riesgo de accidente a 72 h (40 %) y reportes con tope de 10 (15 %).

Nota. El prototipo usa este IPU simplificado de tres criterios para probar las estructuras. La base de datos SIPA calcula el IPU completo con diez criterios; el max-heap funciona igual con cualquiera de los dos,

porque solo compara números.

3. Las estructuras del prototipo

3.1 Lista doblemente enlazada: entrada de reportes

Los reportes llegan en ráfagas y a veces hay que cancelar uno o unir los de otro canal (por ejemplo, los de una app con los de una línea telefónica). En un arreglo, borrar en medio obliga a recorrer todo lo que sigue. Cada nodo de la lista apunta al anterior y al siguiente, así que basta con reconectar dos referencias.

```
def eliminar(self, nodo):
    """Quita un nodo conocido (por ejemplo, un reporte cancelado). O(1)."""
    if nodo.anterior is not None:
        nodo.anterior.siguiente = nodo.siguiente
    else:
        self.cabeza = nodo.siguiente
    if nodo.siguiente is not None:
        nodo.siguiente.anterior = nodo.anterior
    else:
        self.cola = nodo.anterior
    nodo.anterior = nodo.siguiente = None
    self.tamano -= 1
    return nodo.reporte
```

Problema que resuelve	Complejidad	Lo que midió el prototipo
Registrar, cancelar y fusionar reportes sin reacomodar la memoria	O(1) para insertar, eliminar y fusionar	34,945 reportes registrados; la prueba verifica el recorrido en los dos sentidos

3.2 Tabla hash y conjunto: eliminación de duplicados

La clave de un reporte es su ubicación redondeada a cuatro decimales (unos 11 m). Para saber si un bache ya existe, se busca esa clave y sus 8 vecinas, porque dos personas pueden marcar el mismo bache unos metros aparte. Si hay un bache pendiente reportado hace menos de 30 días, es un duplicado; si ya se reparó o el último reporte tiene 30 días o más, se cuenta como una reaparición. Las colisiones se resuelven por encadenamiento (cada cubeta es una lista) y la tabla duplica su tamaño cuando el factor de carga pasa de 0.75. La clase Conjunto se construye sobre la misma tabla y se usa en Dijkstra y en el DFS para marcar los vértices visitados.

```
def _indice(self, clave):
    # Función hash polinomial sobre el texto de la clave (no usamos dict de Python)
    h = 0
    for caracter in str(clave):
        h = (h * 31 + ord(caracter)) % 2_147_483_647
    return h % self.m

def insertar(self, clave, valor):
    """Inserta o reemplaza. O(1) en promedio (amortizado por el rehash)."""
    cubeta = self.cubetas[self._indice(clave)]
    for par in cubeta:
```

```
        if par[0] == clave:
            par[1] = valor
            return
    cubeta.append([clave, valor])
    self.n += 1
    if self.n / self.m > self.FACTOR_MAXIMO:
        self._rehash()
```

Problema que resuelve	Complejidad	Lo que midió el prototipo
Saber en tiempo constante si un bache ya fue reportado	O(1) en promedio; O(n) el rehash, pocas veces	1,064 duplicados y 2,152 reapariciones entre 34,945 reportes en 1.5 s: 314,505 búsquedas en lugar de 611 millones de comparaciones por pares. Factor de carga 0.53; la cubeta más larga tiene 7 elementos

3.3 Árbol AVL y recursividad: consultas ordenadas

Un árbol binario de búsqueda se degrada a una lista si las claves llegan ordenadas, y los reportes llegan ordenados por fecha. El AVL lo evita: después de cada inserción revisa el factor de balance y, si una rama es dos niveles más alta que la otra, la corrige con una o dos rotaciones. La inserción es recursiva: cada llamada baja un nivel y, al regresar, rebalancea el nodo.

```
def _insertar(self, nodo, clave, valor):
    if nodo is None:
        self.n += 1
        return NodoAVL(clave, valor)
    if clave < nodo.clave:
        nodo.izq = self._insertar(nodo.izq, clave, valor)
    elif clave > nodo.clave:
        nodo.der = self._insertar(nodo.der, clave, valor)
    else:
        nodo.valor = valor          # la clave ya existe: se reemplaza
        return nodo

    _actualizar(nodo)
    b = _balance(nodo)
    if b > 1 and clave < nodo.izq.clave:            # caso izquierda-izquierda
        return self._rotar_der(nodo)
    if b < -1 and clave > nodo.der.clave:           # caso derecha-derecha
        return self._rotar_izq(nodo)
    if b > 1 and clave > nodo.izq.clave:            # caso izquierda-derecha
        nodo.izq = self._rotar_izq(nodo.izq)
        return self._rotar_der(nodo)
    if b < -1 and clave < nodo.der.clave:           # caso derecha-izquierda
        nodo.der = self._rotar_der(nodo.der)
        return self._rotar_izq(nodo)
    return nodo
```

El recorrido en orden y la búsqueda por rango también son recursivos. La búsqueda por rango poda las ramas que no pueden contener claves del intervalo:

```
def _rango(self, nodo, minimo, maximo, resultado):
    if nodo is None:
        return
    if minimo < nodo.clave:
```

```
        self._rango(nodo.izq, minimo, maximo, resultado)
    if minimo <= nodo.clave <= maximo:
        resultado.append((nodo.clave, nodo.valor))
    if nodo.clave < maximo:
        self._rango(nodo.der, minimo, maximo, resultado)
```

Problema que resuelve	Complejidad	Lo que midió el prototipo
Mantener los baches ordenados y consultar rangos (una zona, un periodo)	O(log n) para insertar y buscar; O(log n + k) para un rango	Con 34,494 claves, altura 18 (la cota teórica es 21.7) y 14.5 comparaciones por búsqueda, contra 17,942 de la búsqueda lineal

3.4 Max-heap: la cola de atención

Es la estructura que reemplaza a la cola FIFO. Se guarda en un arreglo donde los hijos del nodo i están en 2i+1 y 2i+2, y cada padre tiene una prioridad mayor o igual que la de sus hijos, así que el bache más urgente siempre está en la raíz. Al extraerlo, el último elemento pasa a la raíz y baja hasta su lugar. Además, el heap guarda la posición de cada bache, lo que permite actualizar su prioridad (cuando llega otro reporte o cuando la autoridad la ajusta) sin buscarlo.

```
def extraer_max(self):
    """O(log n): el último pasa a la raíz y baja."""
    if not self.arreglo:
        return None
    self._intercambiar(0, len(self.arreglo) - 1)
    maximo = self.arreglo.pop()
    del self.posicion[maximo[2]]
    if self.arreglo:
        self._bajar(0)
    return maximo

def _bajar(self, i):
    n = len(self.arreglo)
    while True:
        mayor, izq, der = i, 2 * i + 1, 2 * i + 2
        if izq < n and self._mayor(izq, mayor):
            mayor = izq
        if der < n and self._mayor(der, mayor):
            mayor = der
        if mayor == i:
            return
        self._intercambiar(i, mayor)
        i = mayor
```

Para que un bache de prioridad baja no espere para siempre, su prioridad sube un poco con los días de espera, con un tope que impide que rebase a un rojo:

```
def prioridad_efectiva(self, hoy):
    """IPU más envejecimiento acotado: un verde que espera mucho sube, pero no rebasa a un rojo."""
    return round(self.ipu + BETA * min(self.dias_en_espera(hoy), D_MAX) / D_MAX, 4)
```

Problema que resuelve	Complejidad	Lo que midió el prototipo

Entregar siempre el bache más urgente, aunque lleguen reportes nuevos	O(1) ver el máximo; O(log n) insertar, extraer y actualizar	La prueba extrae 500 elementos aleatorios y comprueba que salen en orden descendente; ver los resultados de la sección 5

3.5 Pila: deshacer y rehacer las decisiones de la autoridad

Cuando la autoridad ajusta una prioridad, la decisión se apila con su motivo (es obligatorio). Deshacer saca la última decisión de una pila y la pasa a la otra; rehacer hace lo contrario. Una bitácora aparte guarda todo, incluidos los deshacer, para rendir cuentas.

```
def deshacer(self):
    decision = self.hechas.desapilar()
    if decision is None:
        return None
    self._aplicar(decision["id_bache"], decision["antes"])
    self.deshechas.apilar(decision)
    self.bitacora.append(("DESHACER", decision))
    return decision
```

Problema que resuelve	Complejidad	Lo que midió el prototipo
Corregir un ajuste sin perder el historial	O(1) en la pila, más O(log n) para actualizar el heap	La prueba ajusta, deshace y rehace, y verifica la prioridad en el heap y la bitácora

3.6 Grafo, Dijkstra y DFS: rutas de las brigadas

La red se representa con listas de adyacencia: cada celda de 250 m con baches es un vértice y se conecta con sus vecinas, con la distancia en metros como peso. Dijkstra calcula la distancia mínima desde un origen usando un min-heap propio. El recorrido en profundidad (DFS) es recursivo y sirve para comprobar qué vértices son alcanzables desde la planta de asfalto. En Coyoacán el grafo tiene 952 vértices y 3,624 aristas.

```
def dijkstra(self, origen):
    """Distancias mínimas desde origen. O((V + E) log V)."""
    dist = {v: math.inf for v in self.adyacencia}
    previo = {}
    dist[origen] = 0.0
    visitados = Conjunto()
    cola = MinHeap()
    cola.insertar(0.0, origen)
    while len(cola):
        d, u = cola.extraer_min()
        if u in visitados:
            continue
        visitados.agregar(u)
        for v, peso in self.adyacencia[u]:
            nueva = d + peso
            if nueva < dist[v]:
                dist[v] = nueva
                previo[v] = u
                cola.insertar(nueva, v)
```

```
    return dist, previo

def dfs(self, origen, visitados=None):
    """Recorrido en profundidad recursivo; devuelve el conjunto de vértices alcanzables."""
    if visitados is None:
        visitados = Conjunto()
    visitados.agregar(origen)
    for v, _ in self.adyacencia[origen]:
        if v not in visitados:
            self.dfs(v, visitados)
    return visitados
```

Problema que resuelve	**Complejidad**	**Lo que midió el prototipo**
Ordenar la visita de la brigada para recorrer menos	O((V + E) log V) Dijkstra; O(V + E) DFS	El plan de 8 baches recorre 28.3 km en orden de prioridad y 14.9 km por vecino más cercano: 47 % menos

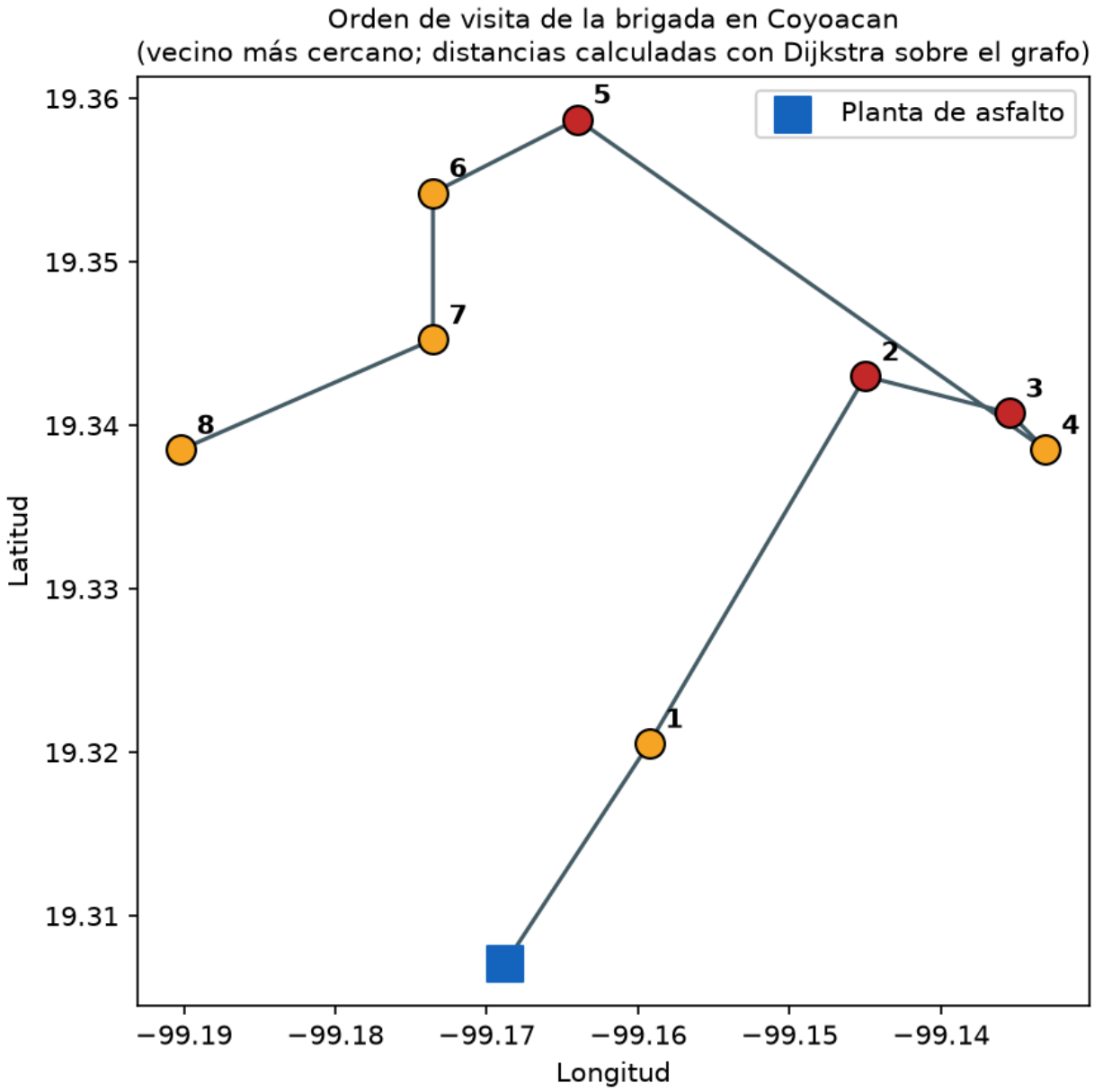


Figura 2. Ruta de la brigada en Coyoacán.

El heap decide qué baches entran al plan del día y el grafo decide en qué orden se visitan, así que acortar la ruta no sacrifica la prioridad.

4. ¿Tabla hash o árbol AVL? La decisión

El incidente pide elegir entre las dos para la búsqueda de coordenadas repetidas. Medimos el tiempo promedio por búsqueda con tamaños crecientes de la base:

Reportes	Búsqueda lineal	Tabla hash	AVL	Comparaciones lineal	Comparaciones AVL	Altura AVL
1,000	19.0 µs	3.4 µs	1.3 µs	522	9.3	12

10,000	194.8 µs	2.4 µs	1.4 µs	4,852	12.6	16
34,945	654.8 µs	8.1 µs	3.4 µs	17,942	14.5	18

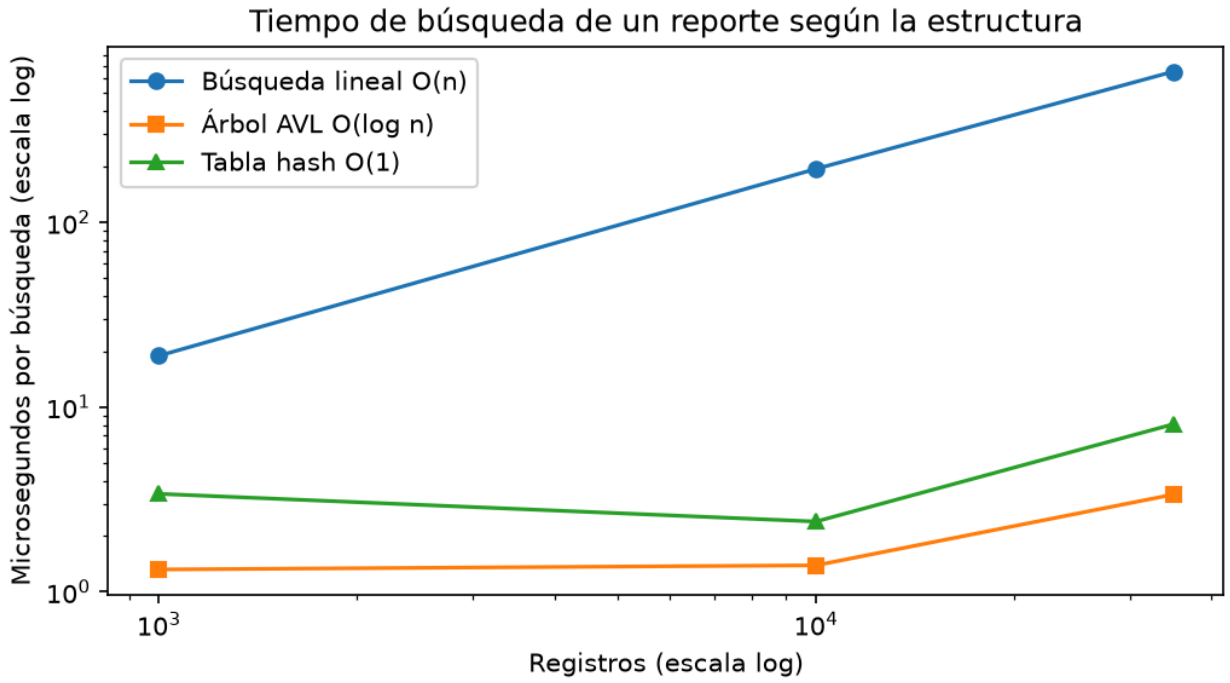


Figura 3. Tiempo de búsqueda según el tamaño de la base.

Las dos estructuras resuelven el problema: con toda la base, la búsqueda lineal tarda 655 µs y la hash y el AVL tardan unos pocos microsegundos. En nuestra implementación el AVL resultó un poco más rápido (3.4 µs contra 8.1 µs), porque la función hash recorre el texto de la clave carácter por carácter en Python. Aun así elegimos la tabla hash para los duplicados, por tres razones:

- Su costo no depende del tamaño de la base: el del AVL crece con log n, y el de la hash se mantiene.
- La desduplicación busca claves exactas (la celda y sus 8 vecinas), que es justo lo que hace bien una tabla hash; no necesita orden.
- Optimizar la función hash es sencillo (por ejemplo, combinar los dos enteros de la clave en lugar de recorrer su texto); el AVL ya está en su mejor caso.

El AVL se queda para lo que la hash no puede hacer: recorrer los baches en orden y consultar rangos. Así cada estructura se usa donde es mejor.

5. Pruebas de correctitud

Las 12 pruebas automáticas (python estructuras/pruebas.py) pasan:

Estructura	Qué se prueba

Lista doblemente enlazada	Insertar, eliminar en medio, fusionar dos listas y recorrer en los dos sentidos
Tabla hash	Buscar, reemplazar un valor y conservar los datos después del rehash
Árbol AVL	Que quede balanceado y ordenado al insertar claves en orden (el peor caso) y al azar
Max-heap	Que siempre extraiga el mayor, que actualizar una prioridad conserve la propiedad de heap y el ejemplo resuelto a mano en el primer avance
Min-heap	Que extraiga siempre la menor distancia (lo usa Dijkstra)
Grafo	Dijkstra contra un ejemplo calculado a mano y la conexión de una cuadrícula
Sistema completo	Que un reporte en la celda vecina cuente como duplicado y uno tras la reparación como reaparición
Pila de decisiones	Ajustar, deshacer y rehacer, con motivo obligatorio y bitácora completa

6. Resultados: FIFO contra la cola de prioridad

Simulamos julio a septiembre de 2019 (90 días, reportes reales) con 15 reparaciones por día y cuatro políticas. La variante con cupo reserva 4 de los 15 lugares diarios para los baches que más han esperado, usando un segundo max-heap ordenado por antigüedad.

Política	**Rojos ≤ 3 días**	**Espera promedio (días)**	**Espera p90**	**Brecha entre alcaldías**	**Pendientes día 90**
FIFO	6 %	21.1	39	6.8	1,355
Cola de prioridad β=0.10	100 %	21.1	55	19.8	1,361
Cola de prioridad β=0.30	100 %	21.1	52	18.5	1,360
Cola de prioridad + cupo	100 %	21.1	51	16.5	1,361

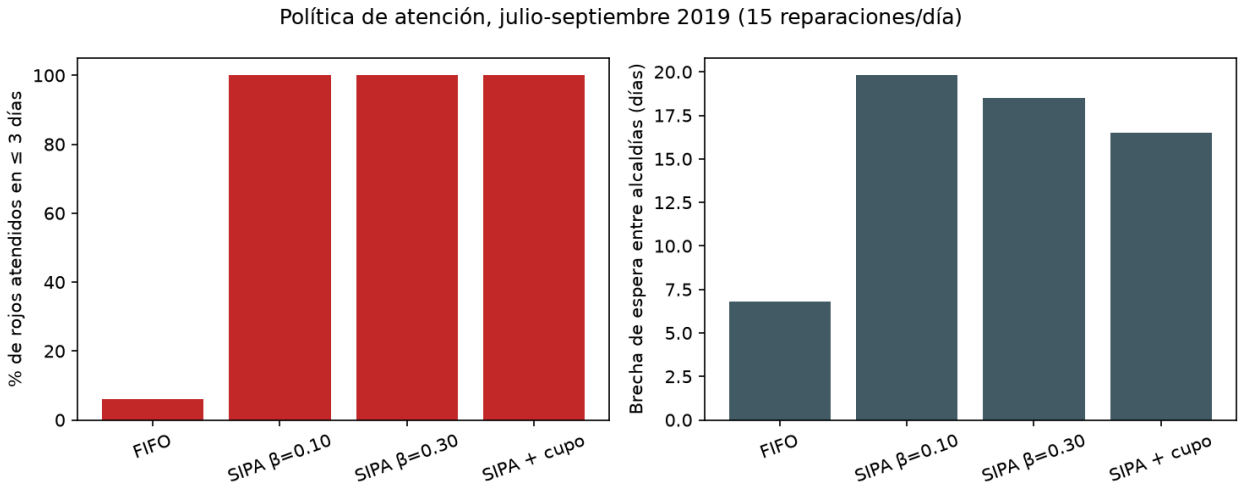


Figura 4. Rojos atendidos a tiempo y brecha entre alcaldías por política.

- La cola de prioridad atiende a tiempo el 100 % de los rojos, contra 6 % con FIFO, sin cambiar la espera promedio (21.1 días): solo cambia quién espera.
- Efecto no deseado: priorizar por riesgo alarga la espera en las alcaldías periféricas, que tienen baches de prioridad baja. La brecha entre alcaldías sube de 6.8 a 19.8 días. El cupo la baja a 16.5 sin perder ningún rojo.
- Ninguna regla de orden evita que la cola crezca si llegan más baches de los que se reparan. Con capacidad suficiente, en cambio, la brecha casi desaparece:

Reparaciones por día	Brigadas	Espera promedio	Brecha	Pendientes día 90
15	3	21.1	16.5	1,361
20	4	14.7	13.0	949
25	5	8.6	6.5	562
30	5	5.2	5.0	287
35	6	2.2	2.6	32

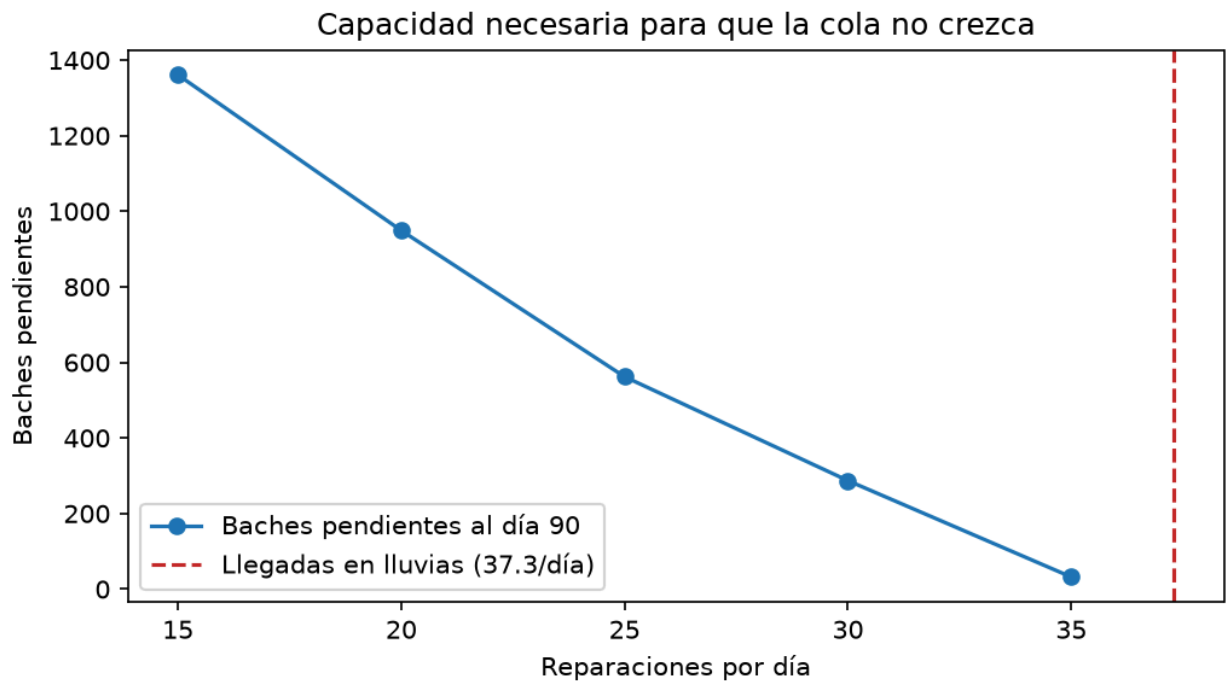


Figura 5. Baches pendientes al final del periodo según la capacidad.

6.1 Prueba de escala con baches reales de Toluca

Para probar las estructuras con datos reales del Estado de México cargamos los 74,319 baches reparados que publica el tablero de bacheo de Toluca (2025-2026) en las mismas estructuras programadas desde cero (estructuras/toluca_ed.py):

Estructura	Resultado
Tabla hash (clave = folio)	inserta todo en 0.283 s; factor de carga 0.567, cubeta más larga 7. Búsqueda: 2.78 µs contra 2,132.8 µs en una lista, 767 veces más rápida
Árbol AVL (clave = fecha, folio)	altura 19 con $\log_2 n = 16.18$ (el límite teórico del AVL es 23.3); balanceado: sí. La consulta por rango de julio a septiembre de 2025 devuelve 5,762 baches en 1.91 ms
Max-heap (profundo primero, después el más antiguo)	74,319 inserciones en 0.198 s; propiedad de heap válida. sorted() de Python tarda 0.028 s porque está escrito en C, pero el heap permite insertar y extraer uno por uno sin volver a ordenar todo cuando llega un bache nuevo

Interpretación. Con diez veces más datos que la simulación de la CDMX, las estructuras se comportan como predice su complejidad: la búsqueda en la tabla hash no crece con n y la altura del AVL crece como log n.

7. Plan de viabilidad

Técnica. El prototipo procesa toda la base (desduplicación de 34,945 reportes) en 1.5 s y simula 90 días en segundos, en una computadora personal. No requiere servidores especiales ni bibliotecas externas para las estructuras.

Operativa. Con 6 reparaciones por brigada al día, la autoridad necesitaría 7 brigadas en lluvias y 4 en secas, y unas 403 toneladas de mezcla al mes en lluvias. SIPA lo recomienda; las brigadas son de la autoridad.

Financiera. Atender los baches abiertos según el semáforo cuesta $4.74 M contra $12.31 M si se espera (reparación más reclamaciones por daños, Contabilidad). Un bache típico cuesta $2,533 si se repara hoy y $3,947 tras 6 meses de lluvias (Cálculo).

Social. El tope de reportes evita que las colonias con más participación digital desplacen a las demás, y el cupo de equidad reduce la brecha entre alcaldías. La autoridad conserva la decisión final y cada ajuste queda en la pila de decisiones con su motivo.

7.1 Contraste entre percepción y evidencia

[PARA EL EQUIPO: escribir aquí, con su fecha real, qué intervenciones consideraba prioritarias el equipo al empezar y por qué. Después, completar la última columna de la tabla con lo que se mantuvo, cambió o se descartó.]

Percepción frecuente	Qué mostró el prototipo	Resultado
El orden de llegada es lo más justo.	FIFO atiende a tiempo el 6 % de los rojos; la cola de prioridad, el 100 %, con la misma espera promedio.	Se descarta
Priorizar resuelve la falta de atención.	Con 15 reparaciones al día la cola crece con cualquier política (1,355 pendientes al día 90). Ordenar no sustituye la capacidad.	Se modifica
Priorizar por riesgo es neutral entre zonas.	La brecha entre alcaldías sube de 6.8 a 19.8 días.	Surge el cupo de equidad
Cada reporte es un bache distinto.	1,064 reportes eran duplicados y 2,152 reapariciones del mismo bache.	Surge la desduplicación y el reencarpetado

8. Propuesta de intervenciones públicas priorizadas

#	Intervención	Estructura que la hace posible	Evidencia	Indicador
1	Atender los baches rojos en 72 horas o menos	Max-heap	100 % de rojos a tiempo contra 6 % con FIFO	% de rojos atendidos en 72 h
2	Eliminar los reportes duplicados al registrarlos	Tabla hash y conjunto	1,064 duplicados detectados en 1.5 s	Reportes duplicados

3	Cupo de equidad: 4 de cada 15 lugares para los baches que más esperan	Segundo max-heap por antigüedad	Brecha entre alcaldías 19.8 → 16.5 días sin perder rojos	Brecha de espera entre alcaldías
4	Rutas diarias por vecino más cercano	Grafo y Dijkstra	47 % menos km en Coyoacán	Km por bache reparado
5	Recomendar a la autoridad ajustar sus brigadas por temporada	Simulación con el max-heap	37.3 baches/día en lluvias; con 15 reparaciones/día quedan 1,361 pendientes en 90 días y con 35, solo 32. Se necesitan 7 brigadas en lluvias y 4 en secas	Pendientes al cierre de mes
6	Reencarpetar las celdas donde el bache reaparece	Tabla hash (reapariciones)	143 celdas con 3 o más reapariciones; la peor (Gustavo A. Madero) tiene 20	Reapariciones por celda al año
7	Registrar y poder revertir cada ajuste de prioridad	Pila de decisiones	Cada ajuste exige motivo y queda en la bitácora	Ajustes con motivo publicado
8	Bacheo preventivo antes de las lluvias	Max-heap con envejecimiento	Esperar 6 meses de lluvias cuesta $1.81 M más (Cálculo)	Costo por bache reparado

Las intervenciones 1, 2, 3, 4 y 7 no requieren presupuesto adicional: solo cambian cómo se organizan los datos y las decisiones. La 5, la 6 y la 8 requieren inversión de la autoridad, justificada en la sección 7 y en el entregable final del problema prototípico.

Celdas candidatas a reencarpetado

Celda	Alcaldía	Baches	Reapariciones	Riesgo 72 h
F249_C108	Gustavo A. Madero	36	20	0.02
F172_C119	Iztacalco	40	12	0.02
F192_C099	Cuauhtémoc	22	9	0.19
F104_C129	Xochimilco	18	11	0.02
F250_C115	Gustavo A. Madero	24	11	0.02
F205_C110	Cuauhtémoc	23	6	0.40
F178_C112	Iztacalco	25	8	0.15
F212_C077	Azcapotzalco	23	8	0.11
F151_C088	Álvaro Obregón	24	5	0.44
F172_C118	Iztacalco	33	9	0.02

9. Limitaciones

- La severidad de los baches es sintética (la base abierta no la trae); el IPU del prototipo es una versión simplificada del de la base SIPA.
- Las rutas usan una cuadrícula de 250 m, no la red de calles real; el siguiente paso es cargar la red vial de OpenStreetMap.
- La productividad de 6 baches por brigada al día es un supuesto que debe calibrar la autoridad.
- Las simulaciones usan 2019; la tasa de llegadas puede ser distinta hoy.
- La función hash recorre el texto de la clave; con una función numérica sería más rápida.

10. Conclusiones

Sustituir la cola FIFO por estructuras adecuadas cambia el resultado sin cambiar los recursos: el max-heap atiende primero lo urgente, la tabla hash evita trabajar dos veces el mismo bache, el grafo acorta las rutas y la pila deja registro de cada decisión. Todas cumplen las complejidades que pide el incidente y lo comprueban las pruebas y las mediciones. La simulación también mostró los límites: priorizar puede dejar atrás a las zonas periféricas, lo que se corrige con un cupo, y ninguna estructura sustituye la capacidad de reparación, que es una decisión de la autoridad.

Referencias

Bader, D. (2020). Common Python data structures (guide). Real Python. https://realpython.com/python-data-structures/

Base maestra de solicitudes de bacheo de la Ciudad de México 2019-2022 [Conjunto de datos]. Proporcionada para el problema prototípico.

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). Introduction to algorithms (4a ed.). MIT Press.

Secretaría de Seguridad Ciudadana de la Ciudad de México. (s. f.). Hechos de tránsito registrados por la SSC [Conjunto de datos]. Portal de Datos Abiertos de la Ciudad de México.

Anexo. Cómo ejecutar el prototipo

Comando	Qué hace
python estructuras/pruebas.py	Corre las 12 pruebas de correctitud
python estructuras/experimentos.py	Mide la eficiencia de búsqueda, desduplica la base, simula FIFO contra la cola de prioridad y calcula la ruta
python estructuras/viabilidad.py	Simula 90 días con las variantes de política y de capacidad